



PAUTA INTERROGACIÓN 1

Pregunta 1

Para cada una de las siguientes consultas, entregue una consulta SQL y, una consulta equivalente en álgebra relacional o en lógica asumiendo la no presencia de duplicados o de nulos. Explique sus consultas. Si la consulta no se puede escribir en álgebra relacional o lógica explique por qué.

1. Todos los pares de especies de pokémon (E_1, E_2) tal que un pokémon E_1 acabaría en un solo ataque (movimiento) con un pokémon E_2 , esto es, su hp quedaría menor o igual a 0.
2. Todas las especies de pokémon que tienen un movimiento inconsistente con su tipo elemental, o sea, la especie tiene un movimiento que no es de su tipo elemental.
3. Una lista con cada tipo elemental junto con el mayor poder de ataque de alguna de las especie de ese tipo elemental.

Solución 1.1

La consulta no tiene una representación en álgebra relacional, ya que no fue vista una forma válida de representar operaciones como la fórmula utilizada para comparar el daño realizado por el primer pokémon al segundo.

Una posible solución para la consulta SQL es la siguiente (existen otras posibilidades válidas de como hacer esta consulta):

```
SELECT DISTINCT p1.name, p2.name
FROM Pokemon p1
      JOIN PokemonMove pm ON p1.pokemonID = pm.pokemonID
      JOIN Moves m ON pm.moveID = m.moveID,
      Pokemon p2
WHERE p2.hp <= (p1.attack * m.power) / p2.defense;
```

Puntaje

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(0.5 puntos)** Indicar fundamentadamente que no hay manera de representar la consulta con álgebra relacional.

- **(1.5 puntos)** Escribir correctamente la consulta en SQL.
- **(-0.5 puntos)** Por cada error importante en la consulta.

Solución 1.2

El primer paso consiste en determinar los tipos de los movimientos de cada Pokémon, la relación entre el identificador del Pokémon y el identificador del tipo de movimiento se obtiene mediante la expresión:

$$\pi_{\text{pokemonID,typeID}}(\mathbf{PokemonMove} \bowtie \mathbf{Moves}).$$

Para aislar los casos donde el tipo del movimiento difiere del tipo del Pokémon, se encuentra la diferencia respecto a la relación PokemonType:

$$\pi_{\text{pokemonID,typeID}}(\mathbf{PokemonMove} \bowtie \mathbf{Moves}) \setminus \mathbf{PokemonType}.$$

Finalmente, para extraer los nombres de los Pokémon que cumplen esta condición, se ejecuta un natural join con la relación Pokemon y se proyecta exclusivamente el atributo name:

$$\pi_{\text{name}}(\mathbf{Pokemon} \bowtie (\pi_{\text{pokemonID,typeID}}(\mathbf{PokemonMove} \bowtie \mathbf{Moves}) \setminus \mathbf{PokemonType})).$$

Por otro lado tenemos la consulta de lógica de primer orden es:

$$Q(p) \leftarrow \exists id_m, n, x, t_1. \text{Moves}(id_m, n, x, t_1) \wedge \text{PokemonMove}(p, id_m) \wedge \neg \exists t_2. \text{PokemonType}(p, t_2) \wedge t_1 = t_2$$

Una posible consulta SQL equivalente es la siguiente (existen otras posibilidades válidas de como hacer esta consulta):

```
SELECT DISTINCT p.name
FROM Pokemon p
      JOIN PokemonMove pm ON p.pokemonID = pm.pokemonID
      JOIN Moves m ON pm.moveID = m.moveID
WHERE NOT EXISTS (
  SELECT 1
  FROM PokemonType pt
  WHERE pt.pokemonID = p.pokemonID AND pt.typeID = m.typeID);
```

Puntaje

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(0.5 puntos)** Indicar la representación en álgebra relacional o en lógica de primer orden de la consulta.
- **(1.5 puntos)** Escribir correctamente la consulta en SQL.
- **(-0.5 puntos)** Por cada error en la consulta.

Solución 1.3

No existe manera de representar esta consulta con álgebra relacional, ya que no fue vista una manera de operar agrupaciones y son necesarias para la consulta.

Una posible solución para la consulta SQL es la siguiente (existen otras posibilidades válidas de como hacer esta consulta):

```

SELECT t.name, MAX(m.power * p.attack)
FROM Pokemon p
      JOIN PokemonType pt ON p.pokemonID = pt.pokemonID
      JOIN Types t ON pt.typeID = t.typeID
      JOIN PokemonMove pm ON p.pokemonID = pm.pokemonID
      JOIN Moves m ON pm.moveID = m.moveID
GROUP BY t.name

```

Puntaje

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(0.5 puntos)** Indicar fundamentadamente que no hay manera de representar la consulta con álgebra relacional.
- **(1.5 puntos)** Escribir correctamente la consulta en SQL.
- **(-0.5 puntos)** Por cada error en la consulta.

Pregunta 2

Considere la siguiente consulta en SQL (sin la presencia de duplicados y de nulos):

```

SELECT p.name, t.name
FROM Pokemon AS p, PokemonType AS pt, Types AS t
WHERE p.pokemonID = pt.pokemonID AND pt.typeID = t.typeID AND
      p.pokemonID NOT IN ( SELECT pm.pokemonID
                           FROM PokemonMove AS pm, Moves AS m
                           WHERE pm.moveID = m.moveID AND m.power = 80 )

```

1. Escriba una consulta equivalente en lógica de primer orden. Explique la correctitud de su consulta.
2. Escriba una consulta equivalente en álgebra relacional. Explique la correctitud de su consulta.

Solución 2.1

Para hacer una consulta en lógica, primero podemos hacer una consulta para obtener todos los pokémones con poder de ataque igual a 80:

$$\begin{aligned}
 E(pokemonID) \leftarrow & \exists moveID, moveName, power, moveTypeID. \\
 & PokemonMove(pokemonID, moveID) \\
 & \wedge Moves(moveID, moveName, power, moveTypeID) \\
 & \wedge power = 80
 \end{aligned}$$

Para finalmente negarlo y obtener:

$$\begin{aligned}
 Q(pokemonName, typeName) \leftarrow & \neg \exists pokemonID, hp, attack, defense, typeID. \\
 & Pokemon(pokemonID, pokemonName, hp, attack, defense) \\
 & \wedge PokemonType(pokemonID, typeID) \\
 & \wedge Types(typeID, typeName) \\
 & \wedge \neg E(pokemonID)
 \end{aligned}$$

Puntaje

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(3 puntos)** La consulta es equivalente y no incluye errores.
- **(2 puntos)** La consulta tiene un error.
- **(1 punto)** La consulta tiene dos errores.
- **(0 punto)** La consulta tiene tres o más errores.

Solución 2.2

Similar a la solución anterior, podemos obtener los pokémones con movimientos de poder de ataque igual a 80 y tomar el complemento, es decir:

$$\begin{aligned} E &:= \pi_{pokemonID}(\sigma_{power=80}(PokemonMove \bowtie Moves)) \\ A &:= \pi_{pokemonID}(Pokemon) \setminus E \end{aligned}$$

Usando la consulta A , podemos escribir la consulta Q como:

$$Q := \pi_{p.name, t.name}(\rho_{name \rightarrow p.name}(Pokemon) \bowtie PokemonType \bowtie \rho_{name \rightarrow t.name}(Types) \bowtie A)$$

Puntaje

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(3 puntos)** La consulta es equivalente y no incluye errores.
- **(2 puntos)** La consulta tiene un error.
- **(1 punto)** La consulta tiene dos errores.
- **(0 punto)** La consulta tiene tres o más errores.

Pregunta 3

Para cada par de consultas diga si las consultas son equivalentes o no en la presencia de nulos. Si son equivalentes, argumente por qué y, si NO son equivalentes, de un contra ejemplo y argumente su respuesta.

- | | |
|---|--|
| 1. Q1 :=
SELECT m.moveID, m.moveName
FROM Moves AS m
WHERE NOT EXISTS
(SELECT 1
FROM PokemonMove AS pm
WHERE pm.moveID = m.moveID) | Q2 :=
SELECT m.moveID, m.moveName
FROM Moves AS m
LEFT OUTER JOIN PokemonMove AS pm
ON m.moveID = pm.moveID
WHERE pm.moveID IS NULL |
|---|--|

<pre> 2. Q1 := SELECT p.pokemonID, p.name, AVG(m.power) FROM Pokemon AS p, PokemonMove AS pm, Moves AS m WHERE p.pokemonID = pm.pokemonID AND pm.moveID = m.moveID GROUP BY p.pokemonID, p.name HAVING COUNT(*) >0; </pre>	<pre> Q2 := SELECT p.pokemonID, p.name, AVG(m.power) FROM Pokemon AS p, PokemonMove AS pm, Moves AS m WHERE p.pokemonID = pm.pokemonID AND pm.moveID = m.moveID GROUP BY p.pokemonID, p.name HAVING COUNT(m.moveID) >0; </pre>
---	--

Solución 3.1

Las consultas son equivalentes.

Q1 selecciona `m.moveID` y `m.moveName` desde `Moves`. La condición `NOT EXISTS` mantiene aquellos movimientos para los cuales **NO** existe una fila en `PokemonMove` con el mismo `moveID`.

Por otro lado, Q2 realiza un `LEFT OUTER JOIN` entre `Moves` y `PokemonMove`. Si un movimiento no encuentra una coincidencia, los atributos provenientes de `PokemonMove` quedan con valor `NULL`. Luego, la condición `pm.moveID IS NULL` selecciona precisamente esas filas.

En presencia de valores nulos, si `m.moveID` es `NULL`, la comparación `pm.moveID = m.moveID` es `UNKNOWN` $\neq$ `TRUE`, incluso si ambos valores son `NULL`. Por lo tanto, dicho movimiento no encuentra coincidencia en ninguna de las dos consultas y es seleccionado por ambas.

Así, Q1 y Q2 retornan los mismos movimientos.

Puntaje

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(0.5 puntos) Equivalencia.** Indica correctamente que ambas consultas son equivalentes.
- **(0.75 puntos) SELECT.** Explica correctamente qué información selecciona Q1.
- **(0.5 puntos) NOT EXISTS.** Explica correctamente que se seleccionan los movimientos para los cuales no existe una coincidencia en `PokemonMove`.
- **(0.75 puntos) LEFT OUTER JOIN.** Explica correctamente que se mantienen todos los movimientos y que aquellos sin coincidencia reciben valores `NULL` en los atributos de `PokemonMove`.
- **(0.5 puntos) IS NULL.** Explica correctamente que esta condición selecciona las filas que no encontraron coincidencia.

Solución 3.2

Las consultas son equivalentes.

La única diferencia entre ambas está en la condición del `HAVING`. En Q1, `COUNT(*)` cuenta todas las filas de cada grupo, mientras que en Q2, `COUNT(m.moveID)` cuenta solamente aquellas filas donde `m.moveID` no es `NULL`.

Sin embargo, antes de realizar el `GROUP BY`, ambas consultas exigen que se cumpla

`pm.moveID = m.moveID.`

Si alguno de estos valores es NULL, la igualdad no es verdadera y la fila es eliminada por el **WHERE**. Por lo tanto, toda fila que llega al **GROUP BY** tiene **m.moveID** distinto de NULL.

Así, para todos los grupos se cumple

$$\text{COUNT}(\ast) = \text{COUNT}(\text{m.moveID}),$$

por lo que ambas consultas retornan exactamente el mismo resultado.

Puntaje

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(0.5 puntos) Equivalencia.** Indica correctamente que ambas consultas son equivalentes.
- **(0.75 puntos) COUNT(*).** Explica correctamente que cuenta todas las filas de cada grupo.
- **(0.75 puntos) COUNT(m.moveID).** Explica correctamente que cuenta únicamente las filas donde **m.moveID** no es NULL.
- **(1 punto) JOIN y valores nulos.** Explica que la condición **pm.moveID = m.moveID** elimina las filas con valores NULL antes del conteo, por lo que **COUNT(*)** y **COUNT(m.moveID)** coinciden.